

Sentinel: An On-Premises Psychophysiological Triage Node for Continuous Mental Health Monitoring

This paper describes the engineering design, security hardening, empirical validation, and deployment economics of Sentinel, a low-cost, on-premises platform for continuous psychophysiological monitoring and discrepancy detection in outpatient mental health care.

Abstract

Mental health care is defined by a monitoring gap: an outpatient sees a psychiatrist for roughly one hour per week and spends the remaining 167 hours unobserved, during which acute stress, panic episodes, or suicidal ideation can develop without intervention. This paper presents Sentinel, an on-premises platform that fuses a subjective channel (patient journal text) with an objective channel (consumer wearable biometrics) through a deterministic, explainable discrepancy detection engine, and surfaces incongruence to clinicians as a prioritized triage queue with an automated crisis escalation protocol.

Sentinel's architecture is organized around four capabilities. First, a three-tier AI pipeline (local OLLaMa inference → cloud Groq → deterministic rule-based fallback) guarantees that journal summarization and risk assessment always return within bounded latency, even during total network outage. Second, a hardware abstraction layer accepts heart rate and heart-rate-variability data from any consumer wearable over REST, WebSocket, or BLE, eliminating vendor lock-in and allowing patients to use devices they already own. Third, a defense-in-depth security model encrypts patient-identifiable fields at rest using Fernet with a key derived from a clinician-entered passphrase (PBKDF2-HMAC-SHA256, 600,000 iterations), hashes passwords with Argon2id, rate-limits the API (in-memory or shared database backend), and maintains a SHA-256 hash-chained audit log. Fourth, a scaling model supports PostgreSQL deployments with cross-worker rate limiting, failover-safe background scheduling via a PostgreSQL advisory lock, and cross-worker real-time push via PostgreSQL LISTEN/NOTIFY.

The system is validated by 222 automated tests plus a 23-case golden-set regression gate enforced in a six-job continuous-integration pipeline. Empirical benchmarks demonstrate 96.0 percent discrepancy-detection accuracy with zero false negatives in ~0.1 ms per profile, constant-time crisis-engine overhead independent of concurrency, and storage I/O that degrades linearly from 10 to 500 patient profiles. The deployment cost model is explicit rather than idealized: approximately 200 USD for a one-time mini-PC, 8–10 USD per year for a domain, and ring provisioning financed through a patient ring-as-a-service subscription tier, priced in a companion business model. The system is designed for clinics with limited technical staff and is validated for deployment without modifying clinical workflows beyond routine monitoring review. The paper presents the complete engineering design, reproducible benchmarks with standardized CSV logbook output, multi-worker scaling verification on real PostgreSQL 17, and a discussion of limitations and future work.

1. Introduction

Mental health disorders represent one of the most significant global health burdens of the 21st century. The shortage of mental health professionals is particularly acute in low- and middle-income countries. In India, the psychiatrist-to-population ratio is approximately 0.75 per 100,000 people, compared to the WHO minimum recommendation of 1 per 10,000 [1]. Approximately 60 percent of districts lack any mental health services whatsoever, creating a fundamental monitoring gap. A patient receiving outpatient care may see a psychiatrist for one hour per week, leaving 167 hours of unmonitored time during which acute stress, panic episodes, or suicidal ideation can occur without intervention. The National Mental Health Survey of India reported that 80 to 85 percent of individuals with mental health conditions do not receive any treatment, and the economic burden of mental health conditions costs the global economy an estimated 1 trillion USD annually in lost productivity. Technology-assisted monitoring offers a scalable alternative that extends the reach of the existing clinical workforce without requiring a proportional increase in the number of clinicians.

Beyond improving patient monitoring, an equally important engineering objective is supporting the sustainability of outpatient mental health practice. Clinical psychologists and psychiatrists often manage large caseloads while balancing therapy sessions, documentation, follow-up, and administrative responsibilities. As patient demand continues to outpace workforce growth, manually reviewing every patient's condition between appointments becomes increasingly impractical. Rather than replacing clinical judgment, Sentinel was designed to automate routine monitoring, prioritize patients showing meaningful psychophysiological discrepancies, and present clinicians with actionable information that helps them allocate their limited time more efficiently. By reducing repetitive monitoring tasks and directing attention toward patients whose subjective and objective channels are incongruent, the platform

addresses the workforce bottleneck directly rather than petitioning for more clinicians.

Consumer wearables provide the objective channel. Devices such as the Oura Ring and Apple Watch have demonstrated research-grade accuracy for heart rate and heart rate variability monitoring. Validation studies report a mean absolute error of 0.36 BPM for the Oura Ring during sleep and intraclass correlation coefficients of 0.91 for HRV compared to medical-grade ECG [3]. The Apple Watch has shown similarly strong performance across multiple independent validation studies, with HRV measurement error within 5 percent of research-grade Polar H10 chest straps. These devices are already in widespread consumer use, with an estimated 200 million wearables shipped globally in 2025 alone and an installed base exceeding 1 billion units worldwide. Among mental health outpatients in high-income countries, wearable ownership is estimated at 30 to 40 percent and growing, suggesting that a significant fraction of the target patient population already owns compatible hardware. This creates an opportunity to leverage existing patient-owned devices rather than requiring clinics to procure and distribute dedicated medical-grade wearables, which would add significant cost and logistical complexity. The approach also avoids vendor lock-in: if a particular wearable model is discontinued or a patient prefers a different device, the HAL can support both simultaneously without any changes to the core discrepancy detection engine or clinical dashboard.

The clinical concept underpinning Sentinel was informed by structured consultations with three practicing clinical psychologists prior to engineering development. All three affirmed the relevance of the monitoring gap and the potential utility of an automated system that cross-references subjective patient journal entries with objective wearable biometric data to flag incongruence. The clinical workflow—patient journal input, biometric ingestion from consumer wearables, automated discrepancy flagging, periodic psychologist review of flagged entries, and escalation of critical alerts—was subsequently reviewed and endorsed by one medical doctor and one additional clinical psychologist. These consultations also confirmed that the clinicians' adoption constraint is not cost of software but cost of operation in time and money: hardware, hosting, and maintenance overhead must be explicit, small, and predictable, which shaped the deployment and pricing model described in Sections 3.11 and 6.

The remainder of this paper is organized as follows. Section 2 situates Sentinel within related work. Section 3 details the system architecture, security model, scaling strategy, and cost of operation. Section 4 presents empirical validation, including a multi-worker scaling verification performed against a real PostgreSQL 17 cluster. Section 5 discusses engineering limitations, addressing the offline requirement identified as a critical gap in existing digital mental health platforms [2]. Section 6 concludes with future work. Section 7 documents post-submission engineering extensions. The paper presents reproducible benchmarks with standardized CSV logbook output and a discussion of limitations.

2. Related Work and Prior Art

Digital mental health monitoring has been approached from several directions in recent literature. Torous et al. [2] provided a comprehensive review of digital mental health platforms and identified data privacy, lack of interoperability, and the absence of offline functionality as critical gaps in existing systems. Their survey of over 1,500 digital health applications found that fewer than 15 percent offered any form of offline capability and fewer than 5 percent used end-to-end encryption for patient data. These findings underscore the need for architectures designed from the ground up for offline operation and data locality.

Wearable-based mental health assessment has been explored in multiple research contexts. The Mindstrong platform demonstrated that smartphone-derived behavioral markers including typing dynamics, scrolling patterns, and social interaction frequency could predict mood changes in patients with major depressive disorder, achieving 82 percent accuracy in mood state classification. The Beiwe platform by Onnela et al. [4] pioneered both active and passive data collection from smartphones for psychiatric research, including GPS-based mobility tracking, phone call and text message metadata, and survey-based self-reports. Both platforms, however, rely on cloud-based processing and do not provide on-premises deployment options for clinics with connectivity constraints or regulatory data sovereignty requirements.

Discrepancy detection as a clinical concept has been studied using ECG and self-report data in controlled laboratory settings. Research by Liao and colleagues demonstrated that mismatches between self-reported anxiety and physiological arousal as measured by skin conductance response predicted treatment outcomes in cognitive behavioral therapy for anxiety disorders. The phenomenon of affective misattribution—where patients report emotional states that differ from their physiological indicators—is well-documented in clinical psychology literature. However, these studies have relied on laboratory-grade equipment and manual or semi-automated analysis. To date, no prior work has automated this discrepancy detection process using consumer wearables and deployed it on low-cost hardware for continuous outpatient monitoring.

On the security front, several frameworks for protecting health data in low-resource environments have been established. The OpenMRS platform provides a reference architecture for offline-capable health information systems used in over 80 countries, demonstrating that local database storage and containerized deployment are viable for clinic-scale applications. The DHIS2 platform has shown secure data collection in rural African clinics with thousands of concurrent users. Neither platform, however, provides real-time biometric ingestion from wearable devices or AI-based triage capabilities. We are not aware of any prior system that combines on-premises deployment, consumer wearable integration via a hardware abstraction layer, rule-based discrepancy detection, AI-based triage, defense-in-depth security hardening, and a multi-worker scaling path in a single publicly available package released for review. A comparison of Sentinel against existing platforms across key dimensions is presented in Section 5.

It should also be noted that several commercial wellness platforms such as Calm, Headspace, and Woebot offer consumer-facing digital mental health interventions at scale. These platforms provide evidence-based therapeutic content but are designed as direct-to-consumer wellness tools rather than clinician-facing monitoring and triage systems. They do not ingest wearable biometric data, do not provide clinician dashboards with aggregated patient views, do not offer on-premises deployment options, and do not perform discrepancy-based triage. Sentinel occupies a distinct category as a clinical support tool for outpatient mental health providers, and its architecture reflects this difference in use case and deployment environment.

The gap in the literature that Sentinel specifically addresses is not the absence of wearable-based physiological monitoring, nor the absence of on-premises health information systems, but rather the absence of these capabilities combined with a clinically motivated discrepancy detection mechanism. Existing systems offer either objective biometric monitoring without subjective patient context (pure physiological tracking) or subjective self-reporting without objective physiological validation (standard patient-reported outcome platforms). The synthesis of both channels through a deterministic rule-based engine that cross-references them in real time, deployable on commodity hardware with explicit, modest operating costs, represents the novel contribution that distinguishes this work from prior art.

3. System Architecture and Methodology

This section describes the engineering architecture and design decisions behind Sentinel. The system is organized into a two-container deployment with a FastAPI backend and React frontend, supported by a hardware abstraction layer for wearable integration, a discrepancy detection engine for cross-referencing subjective and objective data, a three-tier AI pipeline for journal summarization and risk assessment, and a defense-in-depth security model incorporating encryption, network isolation, and tamper-evident audit logging. Production deployments may add an optional PostgreSQL service with a failover-safe scheduler for multi-worker operation. The complete system implementation, including all source code and deployment configuration, is maintained in a version-controlled repository with 222 automated tests that verify each component independently as described in Section 4.

3.1 Deployment Architecture

Sentinel is deployed as a containerized application. The backend container runs a FastAPI application (Python 3.11) using uvicorn as the ASGI server, exposing approximately 120 HTTP routes across 28 routers (authentication, journals, crisis, ring ingestion, triage, follow-ups, bookings, and others). The frontend container runs a React application built with Vite and TypeScript, served by an Nginx reverse proxy that handles TLS termination, static file serving, and route forwarding. The containers communicate over an internal Docker bridge network with the *internal* flag explicitly set to true, meaning the backend has no external network access whatsoever and is only reachable through the Nginx reverse proxy on the frontend container. This zero-trust network topology ensures that even if the backend were compromised through an application-layer vulnerability, an attacker could not pivot to the internet or the local area network. The frontend container is the sole ingress point: it terminates incoming TLS connections, forwards only predefined API routes (*/api/**) to the backend, and returns 404 for all undefined routes.

The storage layer supports two deployment classes. For a single-node clinic deployment, SQLite 3 with Write-Ahead Log (WAL) journal mode provides zero-configuration operation, single-file backup, and no separate server process. For multi-worker or cloud deployment, PostgreSQL is a first-class target: the psycopg2 driver is a declared dependency, schema management is controlled by Alembic migrations (chain `a059d07dd9b6` → `4ff25c97017e` → `d0e1c7a9b2f3` → `9f3e2a1b7c4d`, head `9f3e2a1b7c4d`), and `alembic/env.py` honors `DATABASE_URL` and bootstraps an empty database from the ORM models. All migrations are idempotent and dialect-portable, using SQLAlchemy inspect rather than SQLite-only PRAGMA, and the health write-probe works on both engines. The migration path was verified on a fresh SQLite database and on a real PostgreSQL 17 cluster.

3.2 Hardware Abstraction Layer

Rather than developing custom wearable hardware, Sentinel implements a hardware abstraction layer (HAL) that accepts biometric data from any device exposing heart rate (BPM) and heart rate variability (HRV) via REST, WebSocket, or BLE. The system defines a standard biometric payload format: { bpm: integer, hrv: integer (RMSSD), timestamp: ISO 8601, source: string }. The HAL normalizes device-specific data into this canonical format, enabling hot-swappable device support without modifying the core detection engine. Adapters are defined for the Oura Ring Gen 3 (via Oura Cloud API v2), the Ultrahuman Ring Air (via Ultrahuman REST API), BLE GATT rings, and a mock clinical device that generates synthetic biometric data for automated testing. Adding support for a new wearable device requires implementing a single abstract base class with two methods: *connect()* for authentication and handshaking, and *read()* for retrieving the latest biometric sample in the canonical payload format. Missing data windows are logged but do not trigger alerts, and the system requires at least three consecutive readings within a configurable time window before updating the biometric classification.

This design decision was reached after a failed attempt at custom PCB fabrication using an nRF52840 microcontroller paired with a MAX30102 photoplethysmography sensor for heart rate and HRV measurement. The prototype board, assembled by JLCPCB, suffered from a D+/D- trace routing error on the USB interface that prevented firmware flashing. The failed fabrication attempt consumed approximately six weeks of development time and approximately 400 USD in prototyping and component costs—an explicit, real cost that informed the pivot. The pivot to consumer wearables eliminated hardware certification overhead (FDA 510(k) clearance would have been required for a medical device, and FCC certification for wireless communication), reduced per-patient hardware cost from an estimated 150 USD for a custom device to approximately zero in bring-your-own-device (BYOD) mode, and compressed the deployment timeline from an estimated six months to approximately two weeks. Where clinics choose to provision rings rather than rely on BYOD, the hardware is financed through a ring-as-a-service subscription tier rather than treated as free.

The production HAL is implemented as a pluggable adapter package (app/services/ring) built on a single RingSource base contract with a canonical SensorData payload. Three concrete adapters are implemented: SimulatedRing, which generates deterministic per-user, per-hour biometric streams (calm, balanced, and stressed scenarios) for development and testing; VendorAPIRingSource, an adapter base for vendor SDK/cloud interfaces (Oura, Ultrahuman, and similar); and BLEGATTRingSource, a BLE gateway adapter (bleak) that parses the standard Heart Rate Measurement characteristic (0x2A37), reads battery via the standard battery service (0x180F), and provides configurable characteristic maps and byte parsers for proprietary OEM characteristics. All adapters converge on a single authenticated ingestion endpoint (POST /ring/data), so adding a new device never requires changes to the discrepancy engine, crisis engine, or clinical dashboards.

3.3 Discrepancy Detection Engine

The core detection algorithm is a rule-based heuristic classifier that operates on two independent channels: text sentiment and biometric state. The design is intentionally deterministic rather than machine-learning-based to ensure explainability, auditability, and fully reproducible behavior across different deployments. Every discrepancy decision can be traced back to specific keywords and biometric thresholds, which is a requirement for clinical accountability.

The sentiment channel uses two hardcoded keyword sets (18 positive and 22 negative terms). A negation detection mechanism strips any keyword preceded by one of 26 negation prefixes within a four-word backward window, resolving the classic "I am not happy" false-positive problem that plagues naive keyword-based approaches. The biometric channel classifies the patient's physiological state into three categories based on heart rate and HRV thresholds: high stress (BPM ≥ 110 AND HRV ≤ 25), low stress (BPM ≤ 80 AND HRV ≥ 55), and moderate otherwise. These thresholds are prototype design parameters derived from published norms for resting heart rate (60–100 BPM for adults) and HRV RMSSD distributions (20–70 ms in healthy adults) [5], adjusted to create clear separation between the three classification bands. They are not clinically validated and would require calibration for specific patient populations in a production deployment.

A discrepancy is flagged according to a hardcoded truth table covering all nine sentiment-by-biometric combinations: positive text with high-stress biometrics, negative text with low-stress biometrics, negative text with moderate biometrics, and neutral text with high-stress biometrics. The design prioritizes recall over precision: it is intentionally more permissive in flagging discrepancies, accepting a controlled false-positive rate in exchange for minimizing false negatives in a safety-critical context. The engine was validated against 50 hand-crafted test profiles covering all nine combinations, including edge cases such as empty text strings, zero values for BPM and HRV, crisis-level language, and negation-heavy sentences, and is additionally gated in CI by a 23-case golden set described in Section 4.1.

3.4 Three-Tier Artificial Intelligence Pipeline

Journal summarization and risk assessment follow a three-tier fallback architecture designed to guarantee system responsiveness under all network conditions. Tier 1 attempts inference using a local Ollama instance running Mistral 7B, a 7-billion-parameter language model that runs entirely on consumer hardware with 8 GB of RAM. Mistral 7B was chosen over larger models such as Llama 3 70B because its memory footprint allows it to run on the same mini-PC that hosts Sentinel without requiring a GPU. A threading lock enforces a minimum 500-millisecond gap between consecutive Ollama requests to prevent thundering-herd overload on the host machine. The prompt asks for a structured summary including risk level (low, moderate, high), supporting evidence, and suggested clinical follow-up actions, and explicitly instructs the model to classify any indication of suicidal ideation, self-harm, or imminent danger as high risk regardless of surrounding context.

If Ollama is unavailable, returns a malformed response, or exceeds a 120-second timeout, Tier 2 attempts inference via the Groq Cloud API using the same prompt template against a hosted Mixtral 8x7B model. Groq was chosen over alternatives such as OpenAI or Anthropic for its LPU hardware acceleration, which provides inference speeds under 500 ms per query. API credentials are configured through environment variables, not hardcoded. If both local and cloud AI are unavailable, Tier 3 executes a deterministic rule-based assessment using keyword frequency analysis across five clinical categories, completing in under 50 milliseconds with no external dependencies. This guarantees the system always returns a meaningful risk assessment regardless of infrastructure conditions.

3.5 Encryption, Key Management, and Password Hashing

Patient data at rest is encrypted using an EncryptedText SQLAlchemy TypeDecorator that transparently applies Fernet encryption (AES-128-CBC with HMAC-SHA256 authentication) on every database write and automatic decryption on every read. Fourteen fields across seven models are encrypted (raw journal content, summaries, clinical notes, follow-up descriptions, bookings, triage explanations, and AI analysis fields). The encryption key is derived from a clinician-entered passphrase via PBKDF2-HMAC-SHA256 at 600,000 iterations, producing a 32-byte master key that is split into independent Fernet and HMAC keys via HKDF-Expand with distinct domain separation labels. The key exists only in volatile memory after the unlock step and is never written to disk; a server compromise that exposes the database file, application memory, and filesystem would still not reveal journal contents without the passphrase. If encryption is required (ENCRYPTION_REQUIRED=true) and the key is not available, reads raise instead of leaking ciphertext.

Passwords use Argon2id (argon2-cffi), the memory-hard password-hashing competition winner, with time_cost = 3, memory_cost = 65536 (64 MiB), and parallelism = 4, following OWASP and NIST SP 800-63B guidance [7][8]. Legacy bcrypt hashes are still verified and are transparently re-hashed with Argon2id on the next successful login, so migrating credentials does not force users to reset passwords. This implements a two-factor authentication model at the data layer: identity verification via password (something the clinician knows and enters to log in) and data access via a separate encryption passphrase (something the clinician also knows, entered in a distinct unlock step).

3.6 Security Hardening

A systematic penetration test employing OWASP Top 10 methodology, manual SQL injection testing with crafted payloads, cross-site scripting probes across all user-facing input fields, JWT manipulation attempts, rate-limit bypass tests, and automated scanning with OWASP ZAP identified 22 findings across four severity levels: 4 critical, 7 high, 6 medium, and 5 low. All 22 findings were remediated prior to submission, including a stored XSS vulnerability in the journal entry field (DOMPurify with empty allowlists), the absence of rate limiting on the login endpoint, and a verbose error message disclosing database table schemas (sanitized global exception handlers).

Key mitigations include: session management via HttpOnly, SameSite=Lax, secure cookies for JWT storage, preventing XSS-based token exfiltration; a rate limiter enforced globally (in-memory window default, with a database-backed mode that shares the window across workers); DOMPurify sanitization with empty allowlists on all text-rendering surfaces; sanitized user-facing error messages with server-side stack-trace logging; constant-time HMAC comparisons via hmac.compare_digest; Content Security Policy, HSTS, and nosniff headers served by SecurityHeadersMiddleware; a non-root runtime user in the backend container; and the internal network isolation described in Section 3.1.

3.7 Data Integrity and Audit

All state-modifying operations are logged to a SHA-256 hash-chained AuditLog table. Each row stores the operation type, timestamp, user identifier, affected resource, and the SHA-256 hash of the previous row's concatenated content fields, forming a cryptographic tamper-evident chain: modifying or deleting any historical row breaks the chain by invalidating all subsequent hashes. A dedicated health endpoint recomputes the hash chain from the genesis row and reports whether the chain is intact, enabling automated integrity verification. Database resilience is further

enhanced by a backup function that copies both the database file and its WAL to a timestamped backups directory on startup, alongside a scripted encrypted off-box backup path.

3.8 Frontend Design and User Workflow

The React frontend provides distinct role-based interfaces for patients and clinicians, authenticated through separate login routes with role-specific JWT claims. The patient interface supports daily journal entry via free-text input with a length guideline, visualization of biometric trends over configurable time windows (24 hours, 7 days, 30 days), and a color-coded discrepancy status indicator. The clinician dashboard aggregates all enrolled patients, highlights patients with active discrepancies sorted by severity, displays historical discrepancy frequency patterns, and provides a crisis management interface with a countdown-based escalation workflow with color-coded urgency stages. The frontend uses responsive CSS media queries covering smartphone, tablet, and desktop viewports, and is installable as a progressive web application with a manifest, icons, and a service worker implementing network-first caching for API requests and stale-while-revalidate caching for static assets.

3.9 Testing Methodology and Continuous Integration

The engineering benchmark suite follows IRIS-standard testing conventions. All tests run against an isolated test database with no network dependencies: Tier 1 and Tier 2 AI providers are replaced with mock objects returning configurable responses and latencies. Each test records operation timing, system metadata, and a pass-fail status. The full suite of 222 tests completes in approximately 72 seconds on a Ryzen 5 5600X system with 16 GB RAM.

A GitHub Actions pipeline with six jobs gates every push: lint and typecheck (ruff + tsc), security scanning (safety against declared dependencies), the full backend pytest suite, the frontend production build, Docker image builds for both containers, and the golden-gate regression check (python -m scripts.eval_golden_set --strict), which asserts that the risk and emotion classifiers stay inside pinned confidence bands on the curated golden set.

3.10 Multi-Worker Scaling

The same codebase runs in two deployment classes. Single-node deployments use SQLite and a single API process; multi-worker deployments use PostgreSQL and scale the API to several replicas, with three mechanisms making that scale safe:

Shared rate limiting. `RATE_LIMIT_BACKEND=db` replaces the in-memory limiter with a fixed-window upsert on a `rate_limit_counters` table, so a shared NAT or load-balanced IP is not afforded multiple windows across workers. API replicas run with `RUN_WORKERS=false`.

Failover-safe scheduling. A single scheduler role handles periodic work (email, archiving, surveillance). It acquires a PostgreSQL advisory lock (`SCHEDULER_LOCK_KEY = 749493`) and renews it every `SCHEDULER_HEARTBEAT_SECONDS` (default 10); if the leader process dies, the lock is released by PostgreSQL and a replica acquires it automatically. Verified by killing the leader and observing the standby take over.

Cross-worker real-time push without Redis. WebSocket fan-out across API workers uses PostgreSQL LISTEN/NOTIFY on channel `sentinel_ws` (`WS_PUBSUB=auto/pg/off`): a worker receiving a broadcast publishes a NOTIFY, and all workers, including the one holding the target connection, deliver locally. SQLite deployments keep an in-process fan-out, and crisis state and syncs are always persisted to the database, so nothing is lost if a publish fails.

Schema management for PostgreSQL is provided by Alembic as described in Section 3.1. All scaling behaviors were verified end-to-end against a real PostgreSQL 17 instance (Section 4.6).

3.11 Cost of Operation and Deployment Economics

Sentinel is engineered to be affordable, and its costs are explicit rather than hidden behind a "free" claim. The one-time and recurring cost lines are:

Item	Approximate cost	Type
Mini-PC (Raspberry Pi 5 / Intel N100)	~200 USD	One-time
Cloud VM (if no on-site hardware)	~15 USD/month	Recurring
Domain + TLS	~8–10 USD/year	Recurring
Managed PostgreSQL (optional)	free tier to ~25 USD/month	Recurring

Item	Approximate cost	Type
Groq cloud LLM (optional, triage volume)	cents per month	Recurring
Provisioned smart ring (pilot of 30)	financed via ring/HaaS tier	Recurring (B2B2C)
Patient BYOD	zero marginal hardware	—

The operating software carries no licensing fee, and the AI Tier 3 fallback is built in. The primary ongoing cost is clinician time reviewing flagged discrepancies, estimated at 5–10 minutes per patient per week. The pilot is provisioned from an Emergent Ventures funding award; OEM rings are sourced from Jport (China) and financed through a ring-as-a-service subscription rather than being free. The revenue architecture prices clinic SaaS at ■2,499/month plus ■999/month maintenance, patient tiers at ■499–1,499/month (website access, BYOD, ring/HaaS), and a one-time deployment package at ■9,999–19,999, for a reference unit economy of roughly ■40,948/month recurring per 50-patient clinic at a target gross margin above 70 percent—details in the companion business model.

3.12 Engineering Contribution Summary

The primary engineering contribution is an offline-capable psychophysiological triage architecture that fuses wearable biometric data with patient-reported journal text through a deterministic discrepancy engine, designed specifically for low-resource clinical deployment. The specific contributions are: (a) a negation-aware rule-based classifier achieving 96 percent accuracy with zero false negatives on a 50-profile validation set plus a CI-gated 23-case golden set, (b) a three-tier AI fallback pipeline that guarantees deterministic response times under 50 milliseconds even without network connectivity, (c) a defense-in-depth security model suitable for protected health information on commodity hardware, including Argon2id password hashing, (d) a hardware abstraction layer that supports multiple consumer wearables, (e) a multi-worker scaling path built on PostgreSQL with failover-safe scheduling and LISTEN/NOTIFY fan-out, and (f) a 222-test benchmark infrastructure with standardized CSV logbooks and a six-job CI pipeline.

4. Empirical Validation and Benchmarks

The benchmark suite consists of 222 automated tests spanning authentication, the journal pipeline, discrepancy detection, crisis concurrency scaling, storage I/O performance, AI provider latency, cryptographic operations, ring ingestion, the event store, hardening behavior, and scale-runtime configuration. All tests use the automated harness described in Section 3.9 and produce a standardized CSV logbook with per-run timing, system metadata, and pass-fail status.

4.1 Discrepancy Detection Accuracy

The discrepancy engine was tested against a hand-crafted validation set of 50 profiles spanning all nine sentiment-by-biometric combinations, including edge cases such as empty text strings, zero BPM and HRV values, and negation-heavy journal entries such as "I am not happy" and "no felt not good today." Results:

Metric	Value
Total Profiles	50
True Positives	21
True Negatives	27
False Positives	2
False Negatives	0
Accuracy	96.0%
Precision	91%
Recall (Sensitivity)	100%
Specificity	93%
F1 Score	0.95
Per-Profile Latency	<0.1 ms

Metric	Value
Negation Accuracy	100% (8/8)

The zero false-negative rate is the critical outcome for a safety-critical application. The two false positives arose from biometric readings on the threshold boundary (BPM 109 with HRV 24, and BPM 111 with HRV 26), representing inherent ambiguity in hard-threshold classification rather than a design flaw, and motivating probabilistic classification in future work. Negation-aware preprocessing improved overall accuracy from 92 percent to 96 percent relative to the non-negation-aware baseline.

A golden-set regression gate (23 cases: 16 risk scenarios and 7 emotion scenarios including grief, financial stress, burnout, hopeful recovery, physical illness, and loneliness) pins the risk and emotion classifiers to observed confidence bands and runs with --strict in CI, so the classifier the clinician trusts today is the same classifier tomorrow.

4.2 Cryptographic Performance

PBKDF2 key-derivation latency was measured at five iteration counts on a Ryzen 5 5600X. The chosen value of 600,000 iterations (214.3 ms derive time, 233.2 ms full encrypt–decrypt round-trip) falls within the NIST SP 800-132 recommended range of 100–300 ms for password-based key derivation [6]. Encryption and decryption latency remain below 1 ms at all iteration counts, so the iteration count affects only the one-time derivation step, not per-operation read-write performance. Password hashing with Argon2id adds a comparable one-time cost at login and is also in the OWASP-recommended range.

4.3 Crisis Engine Stress Testing

The thread-based crisis countdown was tested at concurrency levels of 1, 5, 10, and 25 simultaneous simulated patients:

Concurrent Patients	Overhead (ms)	Crisis Detected	Escalation Completed	False Triggers
1	101	Yes	Yes	0
5	101	Yes	Yes	0
10	102	Yes	Yes	0
25	101	Yes	Yes	0

Overhead remained constant at approximately 101 ms regardless of concurrency, confirming $O(1)$ scaling of the singleton crisis state machine. The 20x time-compression testing protocol completed the full escalation in 3,265 ms of real time for 65 simulated seconds, and no simulated patient experienced a missed escalation: the locking mechanism queues escalation events rather than dropping them. Per-patient state machines remain future work for the pilot.

4.4 Storage I/O Benchmarking

SQLite with WAL mode was benchmarked against JSON serialization at 10, 50, 100, and 500 patient profiles:

Profiles	SQLite Read (ms)	SQLite Write (ms)	JSON Write (ms)	Storage (KB)
10	10.2	26.9	6.4	130
50	10.8	52.1	19.7	610
100	11.2	88.4	36.2	1,180
500	11.5	195.2	178.5	5,820

SQLite read latency remained approximately 11 ms across all scales; write latency scaled approximately linearly. Extrapolating, a 30-patient clinic generating three journal entries per patient per day accumulates approximately 33 MB per year—negligible against commodity storage and consistent with 5–10 year regulatory retention requirements. For multi-worker deployments, the PostgreSQL path is covered by the same ORM layer and the same migrations (Section 3.10), with the rate-limiter and pub/sub tables load-tested at 100 concurrent writers with zero data loss.

4.5 AI Provider Latency

The three-tier AI pipeline was benchmarked with simulated provider responses:

Tier	Provider	Avg Latency (ms)	Status
1	Ollama (Mistral 7B)	1,200 (simulated)	Configurable
2	Groq Cloud API	600 (simulated)	Configurable
3	Deterministic Rule-Based	3.2	Guaranteed
Mock (Test Harness)	—	51.5	Passing

The fallback transitions correctly through all three tiers; with Tiers 1 and 2 unavailable, Tier 3 returns a risk assessment in under 5 ms, roughly 375 times faster than simulated Tier 1 and 188 times faster than simulated Tier 2. The two expected benchmark failures (real Ollama and Groq endpoints) occur only in the isolated test environment, where the services are intentionally not running.

4.6 Multi-Worker Scaling Verification (PostgreSQL 17)

Scaling behaviors were verified end-to-end against a real PostgreSQL 17 cluster (not a mock): (a) alembic upgrade head applied the full chain cleanly and env.py bootstrapped an empty database; (b) a two-manager WebSocket pub/sub round-trip delivered a broadcast published on worker A to a connection held on worker B via LISTEN/NOTIFY, and local delivery worked within the same worker; (c) leader failover was verified by acquiring the advisory lock, releasing it, and observing the standby scheduler acquire and heartbeat it; (d) the DB-backed rate limiter correctly shared a single fixed-window budget across processes. A separate 100-user pilot load test hammered /ring/data with retries and offline-buffer bursts: 0 duplicate rows ever landed (2,907 rows / 2,907 distinct sequence IDs), p99 latency of approximately 1.1–1.8 s depending on cadence, and 100 percent success.

5. Discussion and Engineering Limitations

The empirical results demonstrate that a rule-based discrepancy detection engine, combined with deterministic AI fallback and hardware-level network isolation, can produce a consistent triage signal on consumer-grade hardware, and that the architecture scales predictably to PostgreSQL multi-worker deployments without a Redis dependency. The 96 percent accuracy with zero false negatives is particularly relevant for a safety-critical application. However, several limitations must be acknowledged.

First, the keyword-based sentiment analysis, even with negation-aware preprocessing, cannot detect irony, contextual sarcasm, metaphor, clinical jargon, or culturally specific expressions of distress. These are inherent limitations of bag-of-words approaches that contextual language models (Tiers 1 and 2) are designed to address when available.

Second, the biometric thresholds are prototype design parameters, not clinically validated cutoffs, and would require patient-specific calibration in production—likely using the patient's own historical baseline as a dynamic reference.

Third, the singleton crisis state machine can only track one active crisis at a time. The measured 101 ms constant overhead confirms the threading architecture supports per-patient instances, but the state management logic has not yet been refactored from singleton to multi-instance; this is the critical path for the pilot.

Fourth, the cryptographic derivation uses PBKDF2 for the encryption passphrase. The derivation is executed once per process (the derived Fernet key is shared), so it is not a per-request bottleneck; the primary serialization story is documented in Section 3.5. Passwords themselves use memory-hard Argon2id, closing the earlier "planned migration" limitation.

Fifth, absent local AI inference, journal summaries degrade from context-aware natural-language generation to extractive truncation; the discrepancy engine continues to function at full accuracy because it operates independently of the AI pipeline. Clinics deploying without a local Ollama instance lose semantic summarization.

Sixth, clinical validation is limited to pre-engineering consultations with three clinical psychologists, one medical doctor, and one additional clinical psychologist. The system is in the bench prototype phase, validated through in silico testing with simulated profiles; real-world validation with clinical data remains the next milestone, pending institutional ethics board approval, planned for the September 2026 pilot with 30 subjects.

Seventh, the 50-profile validation set, while covering all nine sentiment-by-biometric combinations and multiple edge cases, is hand-crafted and limited in size and diversity. A production-grade validation corpus would require several hundred annotated entries from diverse populations, collected during an approved study. The 23-case golden gate improves regression safety in the interim.

An eighth limitation concerns generalizability to different clinical populations (pediatric patients, cardiovascular comorbidities, beta-blockers), each requiring threshold recalibration and validation data that do not yet exist. Ninth, the system has not undergone formal software certification (IEC 62304, ISO 13485, ISO 14971) or medical-device regulatory review. Tenth, clinician access within the clinic is governed by standard authentication and audit logs; a production deployment would benefit from role-based access logging, automatic session timeout, and integration with clinic identity management.

Relative to the comparison in Section 2, Sentinel occupies the design niche of on-premises deployment plus wearable integration plus offline AI plus discrepancy triage. Its scale target is a 30-patient clinic rather than thousands of users. From a cost perspective, the total hardware for a 30-patient deployment is approximately 200 USD (mini-PC, one-time) plus 8–10 USD/year for a domain, with provisioned rings financed through subscription tiers (Section 3.11); a clinic without on-site hardware can use a cloud VM at approximately 15 USD/month. This compares favorably to cloud platforms charging 10–50 USD per patient per month and to clinical-grade monitors costing 500–2,000 USD per device with limited recording durations, while acknowledging that consumer PPG is less precise than clinical ECG and that coverage is continuous rather than episodic.

6. Conclusion and Future Work

Sentinel demonstrates that an on-premises psychophysiological monitoring platform can be engineered at modest, explicit cost while incorporating security measures appropriate for protected health information. The key contributions are: (1) a negation-aware rule-based discrepancy engine achieving 96 percent accuracy with zero false negatives on the 50-profile validation set and a CI-gated 23-case golden set, (2) a three-tier AI fallback architecture guaranteeing responsiveness regardless of network conditions, (3) a defense-in-depth security model with Argon2id password hashing, encrypted at-rest storage, constant-time operations, network isolation, and input sanitization, (4) a hardware abstraction layer enabling multi-vendor wearables without device-specific code changes, (5) a multi-worker scaling path on PostgreSQL with failover-safe scheduling and LISTEN/NOTIFY fan-out, and (6) a 222-test benchmark suite with standardized CSV logbook output for academic verification, gated by a six-job CI pipeline.

The immediate development roadmap includes: per-patient crisis state machines with independent countdown timers (the critical path for the pilot); a clinical data-collection protocol for approved validation with human subjects, comparing Sentinel's automated classifications against clinician assessments on the same journal and biometric data; a companion mobile application for patient journal entry and wearable synchronization; and uptime/HA hardening for production (encrypted off-box backups, monitoring, certificate rotation).

Long-term, Sentinel is designed for turnkey deployment: a mini-PC pre-loaded with the Docker Compose configuration, requiring only LAN connectivity and clinician credential configuration. The hardware abstraction layer ensures compatibility with any consumer wearable exposing heart rate and HRV data. All source code, benchmark data, engineering logbooks, and technical documentation are maintained in a publicly accessible version-controlled repository released for demonstration and review under an all-rights-reserved license, enabling independent security auditing, community scrutiny, and evaluation of the discrepancy engine for additional languages and cultural contexts.

The total development effort spanned approximately 14 weeks from initial concept to the current prototype, including a six-week failed PCB fabrication attempt (approximately 400 USD of material cost) that informed the pivot to consumer wearables, two clinical validation consultations, core engineering and security hardening, and benchmark infrastructure. The Emergent Ventures funding award supports hardware procurement and pilot deployment; OEM rings sourced from Jport (China) are in progress for the September 2026 pilot with 30 subjects.

7. Post-Submission Engineering Extensions

Following the initial benchmark prototype, Sentinel was extended with five capability areas, all validated through the existing test infrastructure: secured wearable ingestion, a pluggable ring SDK, dual-mode AI output, progressive web application deployment, and the multi-worker scale path described in Section 3.10.

7.1 Secured Wearable Ingestion and Device Binding

Physical wearables must not authenticate with a patient's password. Sentinel implements a device-binding layer backed by a `ring_devices` table. A patient pairs a device serial through `POST /ring/pair`, which returns a one-time device token; only the SHA-256 hash of the token is stored at rest, and tokens are validated with constant-time comparison. Each sensor push supplies `X-Device-Serial` and `X-Device-Token` headers; the ingestion endpoint resolves the owning patient, updates a last-seen timestamp, and records the reading. A revoked device is rejected immediately, and re-pairing a revoked serial re-issues a fresh token. End-to-end verification confirmed correct rejection of wrong tokens, unknown serials, and revoked devices alongside successful authenticated pushes.

7.2 Pluggable Ring SDK

The ring SDK (`app/services/ring`) defines a `RingSource` base contract and a canonical `SensorData` payload, with simulated, vendor-cloud, and BLE GATT adapters as described in Section 3.2. A generic bridge script (`scripts/ring_bridge.py`) runs any adapter on a polling loop and pushes readings through the device-token ingestion path, so the same code base serves simulated demos, BLE rings, and vendor-cloud rings.

7.3 Dual-Mode AI Companion Output

The AI summarization layer generates two distinct outputs from the same journal entry: a warm, supportive summary in the voice of a friendly AI companion for the patient (explicitly avoiding clinical language and unsolicited advice), and a structured clinical summary (OAP format) for the psychologist. Both outputs derive from the same three-tier pipeline, preserving the deterministic fallback guarantees.

7.4 Progressive Web Application

The frontend is an installable progressive web application with a web manifest, installable icons, and a service worker implementing network-first caching for API requests and stale-while-revalidate caching for static assets, reducing reliance on desktop browser access for daily journal submissions.

7.5 Security Batch (September 2026)

Passwords migrated to Argon2id with transparent legacy-bcrypt re-hashing on login (Sections 3.5, 4.2). The read path now fails closed when encryption is required but the key is unavailable. The golden set was expanded to 23 cases and the CI pipeline to six jobs. The suite grew from 216 to 222 tests; `.env.example` was corrected to the production configuration field names so documented variables take effect.

References

- [1] World Health Organization, "Mental Health Atlas 2020," WHO, Geneva, 2021.
- [2] J. Torous, J. Myrick, J. Rauseo-Ricupero, and J. Firth, "Digital mental health and COVID-19: Using technology today to accelerate the curve on access and quality tomorrow," *JMIR Mental Health*, vol. 7, no. 3, 2020.
- [3] A. Kristjansson et al., "Validation of heart rate and heart rate variability measurement using the Oura Ring," *Journal of Medical Internet Research*, vol. 23, no. 2, 2021.
- [4] J.-P. Onnela et al., "Harnessing smartphone-based digital phenotyping to enhance behavioral and mental health," *Neuropsychopharmacology*, vol. 41, no. 1, 2016.
- [5] F. Shaffer and J. P. Ginsberg, "An overview of heart rate variability metrics and norms," *Frontiers in Public Health*, vol. 5, 2017.
- [6] National Institute of Standards and Technology, "NIST SP 800-132: Recommendation for Password-Based Key Derivation," NIST, 2010.
- [7] Open Web Application Security Project, "Password Storage Cheat Sheet," OWASP, 2023.
- [8] National Institute of Standards and Technology, "NIST SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management," NIST, 2017.

AI Utilization Disclosure

This manuscript was edited for syntax refinement, grammar correction, and language formatting using a large language model (Anthropic Claude, accessed via the OpenCode interface). No generative AI was used for the conception of the research question, the design of the engineering architecture, the writing of source code, the analysis of benchmark data, or the formulation of conclusions. The AI tool was used exclusively as a text editor to

improve the clarity and linguistic quality of the authors' original technical writing.